Efficient Semantic Joins

From Trummer's block join to Heterogen v1 and a cheap-to-expensive Heterogen v2 cascade

Paper methods → implementation choices → two controlled experiments

Master 2 project

June 2026

What this presentation explains

Paper

- Why pair-at-a-time semantic joins are expensive
- Tuple, block, optimized, adaptive, and embedding joins
- Simulation and real GPT-4 experiments

Implementation

- How Heterogen v1 maps to the block join
- Why stable structured output was necessary
- What is reused and what is project-specific

Cascade

- Exact-key candidate construction
- Cheap score, thresholds, and expensive fallback
- How models are selected within an experiment

Evidence

- Qwen2.5:3b shared-expensive comparison
- Phi-4-mini shared-expensive comparison
- Cost, routing, precision, recall, and F1

The semantic join problem

Input

Two tables R_1, R_2 and a free-text predicate j.

$$R = \{(t_1, t_2) \in R_1 \times R_2 \mid j(t_1, t_2) = \text{true}\}$$

Traditional join

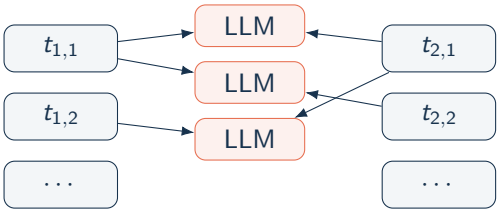
- Predicate is executable code
- Equality, inequality, or deterministic function
- Cheap comparison

Semantic join

- Predicate is natural language
- May require language understanding and common sense
- LLM invocation dominates processing cost

Source: I. Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025.

Paper motivation: tuple nested loops is prohibitively expensive



$r_1 r_2$ calls; static instructions and tuples are repeatedly read

Per-pair prompt

Is predicate j true?
Text 1: t_1
Text 2: t_2
Answer Yes/No.

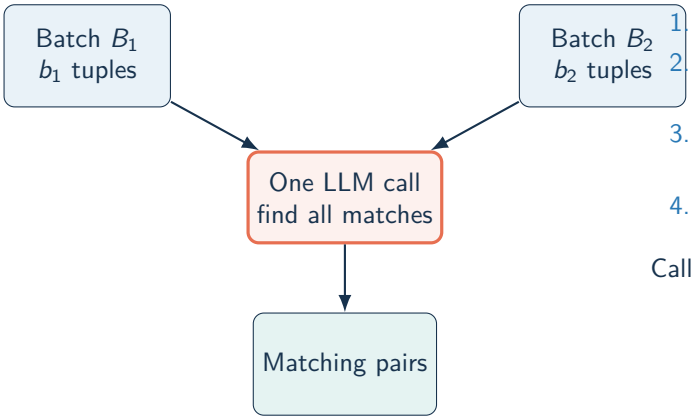
Cost

$$r_1 r_2 (p + s_1 + s_2 + g)$$

The paper estimates that $10{,}000 \times 10{,}000$ entries of 100 tokens could cost roughly \$600k with the cited GPT-4 pricing.

Source: I. Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025.

Core method: semantic block nested loops join



1. Partition each table into batches.
2. For every pair of batches, construct one prompt.
3. Ask the LLM to identify every matching pair.
4. Parse and union the results.

Calls fall from $r_1 r_2$ to approximately:

$$\left\lceil \frac{r_1}{b_1} \right\rceil \left\lceil \frac{r_2}{b_2} \right\rceil$$

Source: I. Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025.

Paper experiment: movie reviews

- Dataset: 50 positive and 50 negative movie reviews.
- Predicate: "Do the two reviews express the same sentiment?"
- High-selectivity workload: many review pairs satisfy the predicate.
- Compared tuple join, block join, adaptive join and embedding baseline.
- Metrics: runtime, token cost, precision, recall and F1.

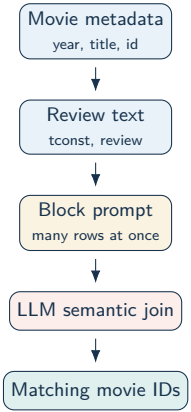
Source: I. Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025.

Paper findings on reviews

- Block joins drastically reduce the number of LLM calls.
- Adaptive joins approach oracle-quality batching without knowing selectivity.
- Accuracy remains comparable to tuple-by-tuple execution.
- Embedding similarity performs worse because sentiment agreement requires semantic reasoning.
- Savings increase as dataset size grows.

Source: I. Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025.

From the paper to Heterogen v1



What questions do we answer?
Natural-language analytical questions over structured movie rows plus unstructured reviews, for example:

Which movies released in 1998 have reviews expressing an overall negative opinion?

How Heterogen v1 answers

1. Put movie metadata and review chunks into blocks.
2. Ask the LLM to apply the full predicate inside each block.
3. Union IDs from all block calls and evaluate against ground truth.

The implementation is therefore a block semantic join adapted to heterogeneous movie metadata + review text.

Heterogen v1 example: one block join

Movie ID	Year	Join key in review	Review signal	Output
`tt0118998`	1998	`tt0118998`	negative / critical opinion	matched
`tt0119196`	1998	`tt0119196`	positive or not clearly negative	rejected
`tt0120735`	1998	`tt0120735`	no matching negative evidence in block	rejected
`tt0133093`	1999	`tt0133093`	negative, but wrong year	rejected
JSON returned by the block join				`["tt0118998"]`

Why Heterogen v2 was introduced

v1 bottleneck

Every block prompt asks one model to:

1. understand two collections,
2. discover identifier matches,
3. enforce year,
4. classify sentiment,
5. return all matching rows.

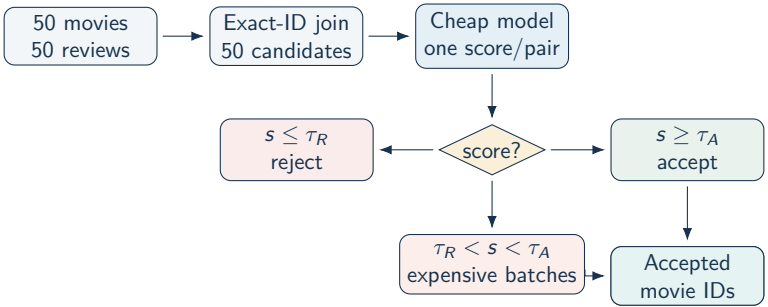
v2 decomposition

Use deterministic structure before semantic inference:

1. exact `movie_id=tconst` candidate join,
2. cheap binary semantic decision,
3. expensive verification for uncertain candidates.

Important: the cascade is a project extension inspired by cost reduction, not an algorithm proposed in Trummer's paper.

Heterogen v2 cascade: complete execution path



Cheap failures fail open to the expensive stage. The expensive model receives explicit candidate pairs, not a cross product.

`project Trummer/heterogen_v2/trummer_join/cascade.py: exact_id_candidates, CascadeJoin.run`

Cheap score and routing in detail

Cheap model output

For each candidate pair:

$$p(1),\ p(0)$$

Score:

$$s = \log p(1) - \log p(0)$$

Example:

$$p(1) = 0.95, \quad p(0) = 0.05$$

$$s \approx 2.94$$

Question: "Does this review express positive sentiment?"

Routing thresholds

$$\tau_R = -1.5, \qquad \tau_A = 3.0$$

- $s \leq \tau_R$: reject
- $s \geq \tau_A$: accept
- otherwise: expensive model

Examples

- $s = -4.6 \rightarrow$ reject
- $s = 2.94 \rightarrow$ verify
- $s = 3.89 \rightarrow$ accept

Expensive fallback in detail

```
Evaluate explicit candidate pairs.
Predicate: year=1998, same movie,
negative review.

PAIR_18:
 Movie: movie_id=tt0118998; ...
 Review: tconst=tt0118998; ...

Return JSON:
 {"matching_pair_ids":[18]}
```

- Uncertain candidates are grouped into batches of 8.
- JSON schema restricts outputs to candidate IDs in the current batch.
- Compatibility parser also recognizes `PAIR_n` labels and unique movie IDs.
- Only expensive-accepted candidates are emitted.

With 37 uncertain candidates:

$$\lceil 37/8 \rceil = 5$$

expensive calls.

`cascade.py: OllamaExpensiveClassifier.classify`

How models are selected in an experiment

```
python3 common_benchmark_v3/scripts/run_all.py \
  --cheap-model ollama/llama3.2 \
  --expensive-model ollama/phi4-mini
```

The orchestrator applies the expensive model to:

- **v1**: all 14 block-join calls
- **v2**: only uncertainty-band batches

`common_benchmark_v3/scripts/run_all.py`

Controlled comparison

- Cheap model fixed: Llama 3.2
- Expensive model shared across v1/v2
- Run A: Qwen2.5:3b
- Run B: Phi-4-mini
- Same 50 rows, predicate, thresholds, and ground truth

Reasoning

- Llama 3.2 is smaller/cheaper.
- Qwen/Phi-4 are candidate expensive verifiers.
- Sharing the expensive model isolates execution strategy.

Local benchmark design

Question

Which movies released in 1998 have reviews expressing an overall negative, critical, or strongly unfavorable opinion?

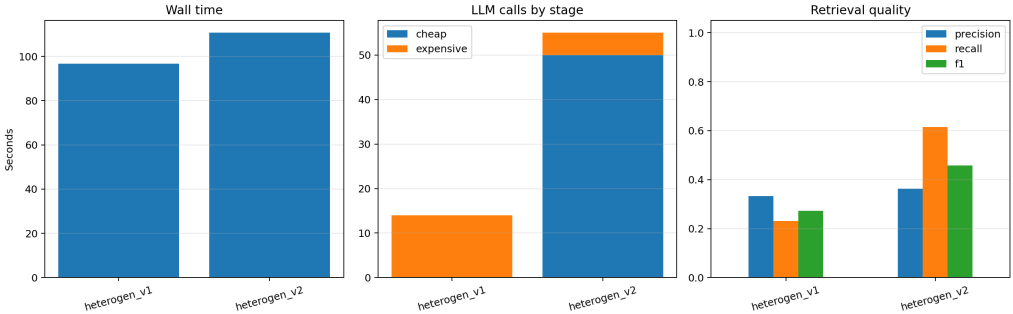
- 50 unique movies and 50 reviews
- 25 movies from 1998; 25 from other years
- Ground truth: 13 movie IDs
- Both implementations receive all rows
- Predicate includes year, exact identifier match, and sentiment

Metrics

- wall time
- total/cheap/expensive LLM calls
- true/false positives and false negatives
- precision, recall, F1
- cascade routing counts

Hardware: local Mac with Ollama; one measured run per configuration.

Experiment 1: Qwen2.5:3b as the shared expensive model



v1: 96.7 s, 14 Qwen block calls, 9 rows, TP=3, FP=6, FN=10, F1=.273.

v2: 110.7 s, 50 Llama + 5 Qwen calls, 22 rows, TP=8, FP=14, FN=5, F1=.457.

Experiment 1 interpretation

v1

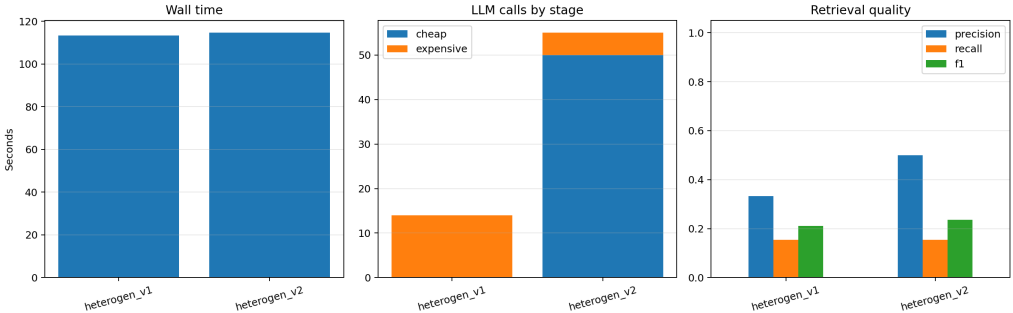
- Fewer calls and lower wall time.
- Large block task remains difficult.
- Stable output fixed parsing, but semantic recall stays low.
- Precision .333, recall .231.

v2

- Cheap stage rejects 13 candidates.
- 37 candidates enter 5 Qwen batches.
- Recall rises to .615.
- Many false positives remain: precision .364.
- Total calls and time exceed v1.

Conclusion: decomposition improves recall/F1, but this cascade is not cheaper in wall time because it pays 50 serial cheap calls.

Experiment 2: Phi-4-mini as the shared expensive model



v1: 113.3 s, 14 Phi-4 calls, 6 rows, TP=2, FP=4, FN=11, F1=.211.

v2: 114.7 s, 50 Llama + 5 Phi-4 calls, 4 rows, TP=2, FP=2, FN=11, F1=.235.

Experiment 2 interpretation

v1

- Stable JSON output prevents zero-row parser failure.
- Precision .333, recall .154.
- Phi-4 still misses most ground-truth rows in large blocks.

v2

- Same routing: 13 cheap rejects, 37 uncertain, 5 Phi-4 calls.
- Precision improves to .500.
- Recall remains .154; only two true positives.
- Wall time is almost identical to v1 despite 55 total calls.

Conclusion: execution strategy alone does not guarantee accuracy; the expensive model's candidate-classification quality remains decisive.

Cross-experiment comparison

Configuration	Time	Calls	Precision	Recall	F1	TP/FP/FN
v1 + Qwen	96.7 s	14 E	.333	.231	.273	3/6/10
v2 + Qwen	110.7 s	50 C + 5 E	.364	.615	**.457**	8/14/5
v1 + Phi-4	113.3 s	14 E	.333	.154	.211	2/4/11
v2 + Phi-4	114.7 s	50 C + 5 E	.500	.154	.235	2/2/11

- Qwen provides the strongest v2 recall and F1 but many false positives.
- Phi-4 is more conservative in v2, improving precision but not recall.
- v1 is consistently cheaper in call count, but large-block semantics limit recall.
- v2 currently optimizes task decomposition, not total serial-call count.

Sources and artifacts

- Immanuel Trummer, "Implementing Semantic Join Operators Efficiently," arXiv:2510.08489, 2025. `https://arxiv.org/abs/2510.08489`
- Local paper: `papers/Implementing Semantic Join Operators Efficiently.pdf`
- Heterogen v1: `project Trummer/heterogen_v1/`
- Heterogen v2: `project Trummer/heterogen_v2/`
- Benchmark: `common_benchmark_v3/`
- Qwen comparison: `outputs/cheap_llama3.2__expensive_qwen2.5_3b/`
- Phi-4 comparison: `outputs/cheap_llama3.2__expensive_phi4-mini/`

Questions?